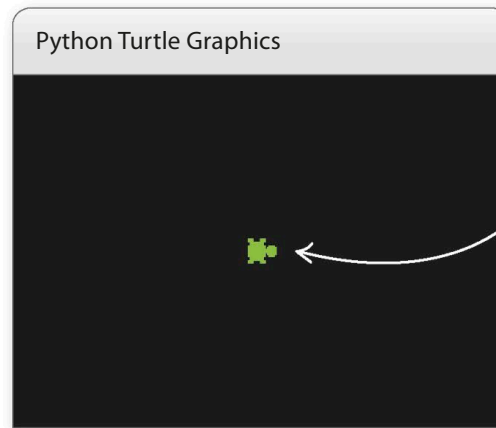


8 Run the project

If you run the code once more, a window will appear after you've entered the line sizes in the shell window. You will now see the turtle. Take a good look at it, because it won't be sitting still for too long!

9 Keep inside the limits!

To stop the turtle from straying, let's set a boundary 100 steps in from the edges of the window. Create this function to check whether or not the turtle is inside the boundary. Type the code shown here under the code in Step 4 and above the code in Step 5.



The turtle starts in the middle of the window.

```
return line_width
```

```
def inside_window():
```

```
    left_limit = (-t.window_width() / 2) + 100
```

```
    right_limit = (t.window_width() / 2) - 100
```

```
    top_limit = (t.window_height() / 2) - 100
```

```
    bottom_limit = (-t.window_height() / 2) + 100
```

```
    (x, y) = t.pos()
```

```
    inside = left_limit < x < right_limit and bottom_limit < y < top_limit
```

```
    return inside
```

```
line_length = get_line_length()
```

With this line, the program gets the turtle's current x and y coordinates.

This is 100 steps to the right of the left edge.

100 steps from the right edge

100 steps from the top

100 steps from the bottom

This sets `inside` to True if the turtle is inside the limits and False if it isn't.

This command passes `inside` back to the code that used this function.

► How it works

The code checks if the turtle's x coordinate is between the right and left limits, and if its y coordinate is between the top and bottom limits.



Don't forget to save your work.

The blue square is shown here to tell you where the limits are set – it won't be visible on your screen.

